

SOFTWARE REQUIREMENTS SPECIFICATION

Gamified Learning App for College Students

A web platform for task-based learning, progress motivation, mentoring, and peer engagement.

Document control	Value
Version	1.0
Date	04 August 2026
Prepared for	College learning-app project
Basis	Project Development Steps (Build Order) supplied by requester
Status	Development-ready draft; confirm assumptions before implementation

How to use this SRS

Use Sections 4-9 as the build contract. The requirements supplied in the source PDF are treated as mandatory; reasonable design details added to make the system buildable are explicitly marked as assumptions or decisions to confirm.

Recommended first action: review the open decisions in Section 3 with your faculty/project team, then start Phase 1 in Section 10.

1. Introduction

1.1 Purpose

This SRS defines the minimum viable gamified learning website for college students. It translates the supplied build order into user-facing behavior, technical constraints, data needs, acceptance criteria, and an implementation sequence.

1.2 Product vision

Students should be able to see what to learn next, complete structured tasks, earn XP and badges, maintain streaks, compare progress with classmates, and ask mentors for help. Administrators should be able to run the learning program without changing code.

1.3 Scope

In scope	Out of scope for v1
Web app for students, mentors, and administrators	Native Android/iOS apps
Tasks/checklists, XP, levels, badges, streaks	Payments, course enrollment, grading system integration
Batch-scoped leaderboard	Public/global leaderboard across all colleges
Mentor requests and one-to-one real-time chat	Video calling or group chat
In-app notifications and email verification	Push-notification mobile app infrastructure

1.4 Glossary

Term	Meaning
XP	Experience points awarded for verified task completion.
Level	A progress tier calculated from the student's total XP.
Badge	A visual achievement unlocked by meeting configured criteria.
Streak	Consecutive days on which a student completes at least one eligible task.
Batch	A cohort of students used to scope the leaderboard.
Mentor	An approved user who may accept requests and chat with assigned students.

2. Users and permissions

Role	Primary permissions
Student	Register and verify email; manage profile; view and complete assigned tasks; earn XP; see own batch leaderboard; request a mentor; chat only with accepted mentor(s); view notifications.
Mentor	Manage mentor profile; accept or reject requests; view linked students; exchange real-time messages;

Role	Primary permissions
	receive related notifications.
Administrator	Create/edit/archive tasks and badges; manage users and roles; publish announcements; review task-completion records; manage mentor access; view operational data.
System	Calculates XP/levels/streaks, awards badges, enforces permissions, and sends notifications.

3. Assumptions and decisions to confirm

The source PDF specifies the feature set but not these product rules. Confirm them before coding to avoid redesign later.

ID	Assumption / proposed rule	Decision needed
A-01	One college deployment, with batches and branches stored per user.	Confirm whether multiple colleges/tenants are needed.
A-02	A student receives XP once per task; an administrator can reset/review a completion.	Choose whether evidence upload or mentor approval is required.
A-03	A daily streak advances when one eligible task is completed in the college time zone.	Set the official time zone and grace-day policy.
A-04	Leaderboard shows rank, display name, level and XP only within the viewer's batch.	Confirm privacy settings and whether students may opt out.
A-05	Level threshold is configurable; initial default is $\text{level} = \text{floor}(\text{total XP} / 100) + 1$.	Confirm XP curve and maximum level.
A-06	Mentors must be approved by an administrator before appearing in search.	Confirm one mentor per student vs. multiple mentors.
A-07	The first release uses email/password login and verified email.	Confirm SSO/college email requirement.

4. Functional requirements

4.1 Authentication and account management

ID	Requirement	Acceptance criterion
FR-AUTH-01	The system shall allow a new student to sign up with name, email, password, batch, and branch.	Valid submission creates a pending account; duplicate email is rejected.
FR-AUTH-02	The system shall hash passwords and never store plain-text passwords.	Database inspection shows only secure password hashes.
FR-AUTH-03	The system shall verify email before enabling protected student features.	Unverified account is redirected to verification prompt.
FR-AUTH-04	The system shall issue a JWT/session after successful	Unauthenticated request to a protected API

ID	Requirement	Acceptance criterion
	login and protect private routes.	returns 401/403.
FR-AUTH-05	The system shall allow a user to sign out and invalidate the active session/client token.	Signed-out user cannot access the dashboard without signing in again.

4.2 Profiles and dashboard

ID	Requirement	Acceptance criterion
FR-PRO-01	Students shall view/edit their profile: name, batch, branch, bio, avatar optional.	Changes persist and are shown after refresh.
FR-PRO-02	The profile shall display total XP, current level, and earned badges.	Values match the database calculation.
FR-DASH-01	The dashboard shall show progress, current streak, recently completed tasks, upcoming/incomplete tasks, and badges.	A seeded student sees all sections and correct values.
FR-DASH-02	The dashboard shall be responsive for phone, tablet, and desktop widths.	Core actions work at 360 px and 1280 px widths.

4.3 Tasks and learning progress

ID	Requirement	Acceptance criterion
FR-TASK-01	Administrators shall create, edit, publish, archive, and assign tasks with title, description, XP value, batch/branch scope, optional deadline, and order.	Published task appears only to matching students.
FR-TASK-02	Students shall view their assigned tasks as a checklist with status and XP value.	Task list loads with pending/completed state.
FR-TASK-03	Students shall mark an eligible task complete.	Completion is stored once, is idempotent, and updates UI.
FR-TASK-04	The system shall prevent a student from completing a task outside their scope or completing it twice.	Invalid API attempts are rejected and no extra XP is issued.
FR-TASK-05	Administrators shall be able to review/revoke a completion with an audit reason.	Revocation recalculates XP/level/streak as applicable and is logged.

4.4 Gamification: XP, levels, badges, streaks

ID	Requirement	Acceptance criterion
FR-GAME-01	The system shall award the configured XP transactionally when a task completion is accepted.	A completion creates one XP ledger entry and increments total XP once.
FR-GAME-02	The system shall derive a student level from total XP using a configurable rule.	Changing XP or the rule updates the displayed level consistently.

ID	Requirement	Acceptance criterion
FR-GAME-03	Administrators shall configure badges with name, image, description, and award condition.	Eligible student automatically receives each badge once.
FR-GAME-04	The system shall calculate and display current and longest streak.	Consecutive eligible days increase current streak; missed day resets current streak.
FR-GAME-05	The system shall notify a student when a level, badge, or configured streak milestone is reached.	Notification appears once with a link to relevant screen.

4.5 Leaderboard

ID	Requirement	Acceptance criterion
FR-LEAD-01	The system shall rank active students by total XP within their batch.	Rank ordering is deterministic: XP descending, then earliest achievement timestamp, then user ID.
FR-LEAD-02	A student shall see only the leaderboard data permitted for their batch.	Cross-batch query is denied; inactive/private users follow confirmed privacy rule.
FR-LEAD-03	Leaderboard shall show rank, display name, level, XP, and optional badge count.	No email, bio, or private data is exposed.

4.6 Mentoring, messaging, and notifications

ID	Requirement	Acceptance criterion
FR-MENT-01	Students shall search approved mentors and submit one request with an optional message.	Duplicate pending request to same mentor is prevented.
FR-MENT-02	Mentors shall accept or reject pending requests.	Student receives status update and notification.
FR-CHAT-01	Accepted mentor-student pairs shall exchange real-time one-to-one messages via Socket.IO.	Both connected users receive the message without page refresh; unauthorized users cannot join room.
FR-CHAT-02	The system shall store message sender, recipient/thread, timestamp, and read state.	Conversation history loads in chronological order.
FR-NOTIF-01	The system shall create in-app notifications for mentor decisions, level-ups, and streak milestones.	Unread count and read state update correctly.
FR-NOTIF-02	Administrators shall publish announcements to a chosen audience.	Selected students see one notification/announcement entry.

4.7 Administration

ID	Requirement	Acceptance criterion
FR-ADMIN-	Administrators shall manage users, roles, mentor	Each action is restricted to admin accounts and

ID	Requirement	Acceptance criterion
01	approval, task catalog, badges, and announcements.	persists correctly.
FR-ADMIN-02	Administrators shall view basic metrics: active users, completions, XP awarded, pending mentor requests.	Metrics match seeded database counts.
FR-ADMIN-03	The system shall retain an audit log for role changes, task edits, XP adjustments/revocations, and badge configuration.	Log captures actor, action, target, timestamp, and before/after summary.

5. Business rules

1. Task completion, XP award, level recalculation, and badge evaluation shall run in one database transaction or reliable queued workflow so duplicate XP cannot occur.
2. Only published tasks with matching batch/branch scope may be completed by a student.
3. XP values and level thresholds are administrator-controlled configuration, not hard-coded UI values.
4. A badge is awarded at most once per user unless its configuration explicitly supports repeat awards.
5. A mentor-student chat room exists only after the mentor request is accepted and becomes inaccessible when that relationship is disabled.
6. All timestamps shall be stored in UTC; streak logic shall use the approved college time zone.

6. Data requirements

The following core tables come directly from the supplied project outline. Two supporting tables are recommended to preserve XP and badge-award history.

Table	Key fields / purpose
users	id, name, email (unique), password_hash, role, email_verified_at, batch, branch, bio, avatar_url, total_xp, level, status, created_at
tasks	id, title, description, xp_value, batch_scope, branch_scope, due_at, sort_order, status, created_by, created_at
user_tasks	id, user_id, task_id, status, completed_at, reviewed_by, review_reason; unique(user_id, task_id)
badges	id, name, description, image_url, criteria_type, criteria_value, active, created_by
user_badges (recommended)	id, user_id, badge_id, awarded_at; unique(user_id, badge_id)
xp_ledger (recommended)	id, user_id, source_type, source_id, amount, reason, created_at; unique source prevents duplicate awards
mentor_requests	id, student_id, mentor_id, note, status, responded_at, created_at
messages	id, mentor_request_id/thread_id, sender_id, body, read_at, created_at
streaks	user_id, current_streak, longest_streak, last_qualified_date, updated_at
notifications	id, user_id, type, title, body, link, read_at, created_at

Table	Key fields / purpose
announcements (recommended)	id, title, body, audience_scope, published_at, created_by

6.1 Relationships and integrity

- One user has many task records, XP ledger entries, notifications, badges, and messages.
- One task has many user task records; user_tasks must be unique per student-task pair.
- A mentor request links one student and one mentor; its accepted state authorizes messaging.
- Use foreign keys, indexed IDs, unique email, unique user-task, and unique user-badge constraints.
- Soft-delete/archive configuration records rather than deleting records that affect historical XP or reports.

7. External interfaces and architecture

7.1 Proposed architecture

Layer	Recommended responsibility
Frontend	React single-page app; responsive views; React Router; authenticated API client; Socket.IO client.
Backend	Express REST API; JWT middleware; role-based authorization; Socket.IO server; validation layer; services for gamification.
Database	PostgreSQL with migrations, transactions, constraints, indexes, and backups.
Email	Transactional provider for verification and optional notification emails.
Deployment	Frontend static host; API service host; managed PostgreSQL; secure secrets; HTTPS; centralized logs.

7.2 API outline

Area	Example endpoints
Auth	POST /auth/signup, POST /auth/login, POST /auth/verify-email, POST /auth/logout, GET /auth/me
Profile	GET /users/me, PATCH /users/me
Tasks	GET /tasks, POST /tasks/:id/complete, GET /admin/tasks, POST/PATCH /admin/tasks/:id
Gamification	GET /progress/me, GET /leaderboard?batch=me, GET /badges
Mentoring	GET /mentors, POST /mentor-requests, PATCH /mentor-requests/:id
Messages	GET /conversations/:id/messages; Socket.IO events: join_conversation, message:send, message:new
Notifications	GET /notifications, PATCH /notifications/:id/read
Admin	GET /admin/users, PATCH /admin/users/:id, CRUD /admin/badges, CRUD /admin/announcements, GET /admin/metrics

8. Non-functional requirements

Category	Requirement / target
Security	HTTPS in production; bcrypt/Argon2 password hashing; JWT expiry/rotation policy; server-side role authorization; input validation; rate limiting on auth; parameterized SQL/ORM; no secrets in source control.
Privacy	Collect minimum personal data; do not display emails on leaderboards; give administrators role-limited access; define retention/deletion policy with college.
Performance	Typical dashboard/task/leaderboard API response under 500 ms at expected pilot load; paginated list endpoints; indexed leaderboard query.
Reliability	Transactional task completion and XP awarding; daily database backup; error logging; user-friendly retry/error states.
Usability	Responsive UI, clear completion confirmation, readable contrast, keyboard-operable controls, labels for form fields, empty states.
Compatibility	Latest two versions of Chrome, Edge, Firefox, Safari; mobile browsers at 360 px minimum width.
Maintainability	TypeScript recommended, migrations, environment-based configuration, linting, automated tests, API documentation, README.
Observability	Structured API errors, request IDs, authentication/authorization failure logs, health endpoint, monitoring alerts for service/database failures.

9. Testing and acceptance plan

Test layer	Minimum coverage
Unit tests	XP calculation, level thresholds, badge rules, streak rules, validation, permission helpers.
API/integration tests	Signup/login/verification, protected routes, task completion idempotency, transactions, leaderboard scoping, mentor request transitions.
Socket tests	Only accepted mentor/student pair can join a conversation; message persistence and delivery; reconnect behavior.
UI/e2e tests	Student completes task and sees XP/level update; mentor accepts request; chat works; admin publishes task/announcement.
Security tests	Authorization matrix, invalid JWT, SQL injection attempts, XSS-safe rendering, rate limit checks.
UAT	Pilot with representative student, mentor, and administrator; execute acceptance criteria in Section 4.

Release acceptance: all P0 flows pass, no known authorization bypass or duplicate-XP defect remains, production environment passes smoke test, backups are verified, and admin user guide is available.

10. Step-by-step development plan

Phase	What to do	Done when
0. Confirm product rules	Resolve Section 3 decisions; name roles; choose visual identity; define initial task list, XP curve, badge rules, and batch structure.	Signed-off rules and seeded sample data are ready.
1. Set up project	Create GitHub repo; initialize React frontend and Express backend; set up PostgreSQL; add .env.example; add linting, formatting, branches, issue board, and CI.	Frontend, API, database migration, and health check run locally and in CI.
2. Design database	Create migrations for users, tasks, user_tasks, badges, mentor_requests, messages, streaks, notifications plus user_badges/xp_ledger. Add constraints and indexes.	Migration runs from empty database and seed script creates demo users/tasks.
3. Build auth and roles	Implement sign-up, login, hashed password, JWT/session, email verification, auth middleware, and Student/Mentor/Admin authorization.	Protected-route and role tests pass.
4. Build student base	Create profile edit/view and dashboard shell; display account data, progress placeholders, and responsive navigation.	A signed-in student can edit profile and reach all allowed screens.
5. Build task workflow	Create admin task CRUD, student checklist, task scope filtering, completion/review endpoint, and audit events.	Student sees only eligible tasks and duplicate completion is impossible.
6. Build gamification engine	Add XP ledger, total XP update, level calculation, badge rule evaluation, streak service, and notification triggers.	Automated tests prove each calculation and rollback case.
7. Complete dashboard and leaderboard	Show real progress, streak, recent tasks, badges; write indexed batch leaderboard query and UI.	Dashboard matches records; leaderboard is scoped and deterministic.
8. Add mentoring	Create mentor profiles, discovery, request/accept/reject flows, relationship visibility, and notifications.	Only approved mentors are searchable; state changes notify users.
9. Add real-time chat	Configure Socket.IO authentication, private rooms, message persistence, read status, and reconnect/error handling.	Accepted pairs can chat live; unauthorized connection is blocked.
10. Build	Finish user/role management, task/badge/announcement administration, metrics, and audit log views.	Admin can operate all v1 content without database edits.

Phase	What to do	Done when
admin operations		
11. Test and harden	Run unit, API, e2e, accessibility, load-smoke, and security tests; fix P0/P1 issues; perform pilot UAT.	Release acceptance criteria pass.
12. Deploy and document	Deploy frontend/API/database; configure HTTPS, secrets, domain, backups, logging, and monitoring; write README, architecture diagram, screenshots, and demo video.	Production smoke test and rollback plan are verified.

11. Suggested initial backlog (first two sprints)

Priority	Backlog item	Why it comes first
P0	Repository, environments, CI, database migrations, seed data	Enables safe parallel development and repeatable testing.
P0	Authentication, email verification, role-based middleware	Every protected feature depends on identity and permissions.
P0	Admin task CRUD and student task checklist/completion	Delivers the core learning loop.
P0	XP ledger, level rule, dashboard progress	Makes the core loop gamified and visible.
P1	Badges, streaks, batch leaderboard	Adds motivation after core completion is reliable.
P1	Mentor requests, notifications, real-time chat	Adds support/community once user relationships are secure.
P1	Admin metrics/audit, deployment hardening	Needed before pilot release.
P2	Rich analytics, evidence uploads, SSO, mobile app	Consider only after pilot feedback.

12. Risks and mitigations

Risk	Mitigation
Duplicate or incorrect XP	Use unique constraints and transactions; calculate from ledger; test concurrent completion.
Leaderboard discourages some learners	Keep it batch-scoped; consider opt-out or personal-progress view; seek pilot feedback.
Chat privacy/safety concerns	Limit rooms to accepted relationships; allow block/report; establish moderation and retention policy.

Risk	Mitigation
Unclear badge/streak rules	Confirm rules before launch; configure them in admin rather than hard-code.
Scope grows too quickly	Release P0 core loop first; keep P2 ideas out of initial deployment.
Student data exposure	Apply least privilege, protect APIs server-side, avoid public profile details, and review college policy.

13. Final pre-development checklist

- Approve roles, batch/branch structure, and privacy policy.
- Choose the initial 20-30 tasks, their XP values, and completion-review policy.
- Approve level thresholds, 5-10 badge rules, and streak definition.
- Choose email provider, hosting, domain, and production database provider.
- Create wireframes for sign-up, dashboard, task list, leaderboard, mentoring, chat, and admin screens.
- Create the GitHub project board from Section 10 and assign one owner per phase.
- Start with a small pilot batch before opening the platform to all students.